RAJALAKSHMI ENGINEERING COLLEGE
Approved by AICTE | Affiliated to Anna University | Accredited by NAAC

VII Semester B. Tech AIDS
AD23731 — Foundations of Agentic AI
Academic Year: 2026-2027 (ODD) Batch: 2023-2027
Faculty: Dr. A. Beulah

SignAlert: An Agentic Vision Framework for Real-Time Detection and Escalation of Medical Distress Signs in Indian Sign Language

Team Members:
Sakthi Mukesh T — 231801145
Sathvikha V — 231801160

PROJECT REVIEW II — FUNCTIONAL PROTOTYPE, ORCHESTRATION AND PRELIMINARY EVALUATION REPORT

Recap of Objective and Scope

SignAlert is an agentic computer-vision system that continuously observes a monitored space, recognises a curated vocabulary of Indian Sign Language (ISL) distress signs, and autonomously carries a confirmed event through a structured, multi-stage caregiver-escalation pipeline rather than merely raising a passive alert. Project Review I established the problem statement, stakeholder analysis, and the agent architecture and tool selection for this system. This report documents the deliverables for Project Review II: the functional prototype now implemented, the agent orchestration and state-management design realised in code, the tools and services integrated into the working pipeline, the retrieval-and-generation pattern used by the reasoning layer, and preliminary evaluation results benchmarked against comparable systems, grouped below under five general headings that together cover all seven expected deliverables for this review.

1. System Implementation and Functional Prototype

The functional prototype implements the complete perception-to-action loop end to end, organised into five cooperating layers that move from raw visual input to a real-world action: a camera feed, a perception module (MediaPipe Holistic landmark extraction feeding an LSTM sequence classifier), a pose-based fallback detector, a confirmation layer, an escalation controller, and a dispatch layer that delivers notifications through Twilio. Each layer is developed and tested as an independent component, so that no single layer's reliability has to be assumed for the system as a whole to be evaluated.

On the perception side, each video frame is processed by MediaPipe Holistic to yield 33 pose landmarks (x, y, z, and visibility) together with 21 left-hand and 21 right-hand landmarks (x, y, z), concatenated into a 258-dimensional feature vector per frame. Because clips vary naturally in length, 30 evenly spaced frames are sampled from every clip using linear interpolation over frame indices, producing a uniform (30 × 258) input tensor regardless of whether the original recording ran two or four seconds. This landmark-based representation, rather than raw pixels, is what keeps the prototype trainable on a small, purpose-recorded dataset while remaining robust to lighting, clothing, and camera-angle variation.

The backend is implemented in FastAPI, chosen specifically for its native asynchronous support and WebSocket capability, which the prototype needs for continuous, low-latency streaming of camera frames from the client and continuous delivery of live prediction results back to it. Incoming frames are accumulated into a rolling buffer; once 30 frames are available, MediaPipe processing and LSTM inference run on the buffered sequence, after which the buffer slides forward so that predictions keep updating for as long as the monitoring session remains active. The frontend is a lightweight web client that captures webcam frames, streams them to the backend over the open WebSocket connection, and renders the most recently predicted label alongside the current agent and escalation status, giving a caregiver or tester a live view into what the system is currently observing.

In its current state, the prototype recognises the five-sign distress vocabulary (Heart-pain, Help, Pain, Cant-breathe, Dizzy) end to end from a live or recorded camera feed, runs the pose-based fall and stillness fallback in parallel, applies the confirmation logic before anything is escalated, and dispatches real SMS notifications through Twilio operating in trial-account mode, which restricts delivery to caregiver numbers that have been pre-verified — an acceptable constraint at this project stage, given the small, known set of contacts used for demonstration and testing.

To validate the prototype without depending on continuous access to a live camera and willing signers, a synthetic event simulator was also built alongside the live pipeline. The simulator replays labelled distress-sign and fall/stillness sequences directly into the perception and fallback interfaces, which lets every downstream agent — confirmation, risk assessment, escalation, and dispatch — be exercised and regression-tested independently of camera availability. This was particularly useful for testing edge cases that are inconvenient to reproduce live, such as a confirmation window that expires without a cancel gesture, a rapid sequence of repeated high-confidence detections meant to trigger the panic override, or a caregiver contact that never acknowledges a notification within its timeout. Running these scenarios through the simulator before testing them against the live camera feed materially reduced the number of end-to-end debugging cycles needed once the full pipeline was assembled.

2. Agent Orchestration, Workflow, and State Management

The system's agent split was carried over from the architecture defined in Review I and is now implemented as a working orchestration graph. Each agent has a narrow, well-defined responsibility, summarised below.

Agent	Purpose	Input → Output
Perception	Converts video into structured sign/pose data	video frame → {sign_id, confidence, pose_state, timestamp}
Fallback Detector	Watches for falls or prolonged stillness independent of signing	pose sequence → {event_type: fall/stillness, confidence}
Confirmation	Filters false positives before anything is escalated	raw detection → {confirmed: true/false} after buffer window and optional cancel gesture
Risk Assessment	Reasons about severity from the detected sign and context	confirmed event → {severity, recommended_stage, reasoning}
Escalation Controller	Drives the fixed 5-stage escalation ladder, advancing on timeout	severity → {current_stage, contacts_notified, timeout_status}
Dispatch	Delivers notifications and the final emergency description	stage + contact list → delivery confirmation
Orchestrator	Routes state between agents; one instance per monitored room	—

Orchestration is implemented with LangGraph, chosen because the escalation workflow has explicit stages and conditional branching rather than open-ended, free-form multi-agent dialogue — a state-machine-oriented framework is a closer structural fit than general-purpose multi-agent chat frameworks such as CrewAI or AutoGen. Formally, the escalation process is a finite-state sequence S

= (s0, s1, …, sn), where each transition si → si+1 is gated by either an explicit timeout τi elapsing without human acknowledgment, or an immediate override triggered by a repeated, high-confidence panic gesture. One LangGraph instance is instantiated per monitored room, so state for concurrent rooms never mixes.

The Risk Assessment agent is where the workflow's reasoning and self-correction behaviour lives. Rather than escalating on a single raw detection, it looks at the confirmed event alongside recent context — whether the same sign has been repeated, whether a panic gesture has been given, and whether an earlier confirmation prompt in this session went unanswered — before recommending a severity level and an escalation stage to the controller. This is also where the system corrects for an important failure mode: a single low-confidence or borderline detection is not enough to advance the escalation ladder on its own, but the same signal repeated across the confirmation buffer, or reinforced by an explicit panic override, is treated as sufficient grounds to skip ahead rather than wait through the full multi-minute caregiver timeout. This reflective step — checking accumulated evidence rather than reacting to the first frame — is what keeps the ladder both fast enough to be useful in a genuine emergency and resistant to a single noisy prediction triggering an unnecessary escalation.

Memory and state management centre on a per-room state object maintained by the orchestrator. This state tracks the current escalation stage, the list of contacts already notified, the countdown/timeout status for the active stage, the rolling confirmation buffer of recent frame-level predictions, and a cooldown timer enforcing a minimum 30-second gap between successive triggered actions so that fluctuating predictions during an ongoing event cannot flood a caregiver with duplicate notifications. The most safety-critical property of this state design is its independence from the client connection: once an action has been triggered, the escalation process is decoupled from the state of the camera or monitoring session, so that if a user or caregiver stops live monitoring on the front end, any in-flight escalation stage and its cooldown continue running to completion on the backend. This means that closing the dashboard or losing the camera connection — intentionally or accidentally — cannot silently cancel an emergency response that is already underway, which is a deliberate design choice validated during this review.

3. Tool and Technology Integration

Each architectural layer defined in Review I has now been wired to a concrete tool or service in the working prototype. The table below summarises what is integrated and why each choice was retained through implementation.

Layer	Tool / Service	Integration Role
Perception	MediaPipe Holistic + TensorFlow/Keras LSTM	Frame-by-frame landmark extraction feeding a trained sequence classifier for the 5-sign vocabulary.
Fallback Detection	Custom pose-heuristics module	Geometric rules over pose landmarks flag sudden vertical drops (falls) or prolonged stillness.
Orchestration	LangGraph	State-machine graph coordinating Perception, Fallback, Confirmation, Risk Assessment, Escalation, and Dispatch agents per room.

Layer	Tool / Service	Integration Role
Reasoning / Summarisation	DeepSeek API / Qwen3 via Ollama	Converts a confirmed event and its context into a clear text description; falls back to a fixed template if no API key is configured.
Backend	FastAPI + WebSocket	Async server streaming camera frames in and prediction/status updates out with low latency.
Frontend	Single-file HTML/CSS/JS dashboard	Captures webcam frames, streams over WebSocket, and displays live status, predictions, and alert history.
Notifications	Twilio (SMS API, trial mode)	Delivers caregiver and secondary-contact notifications; restricted to pre-verified numbers at this stage.

The integration pattern deliberately keeps every external service optional at the boundary rather than load-bearing for the safety logic itself: if Twilio credentials or an LLM API key are absent, the system substitutes a clearly labelled simulated SMS/push log and a templated text description respectively, so the escalation state machine and its timeouts can still be demonstrated and tested end to end without a paid account. This separation of "what must always work" (the state machine, the confirmation logic, the timeouts) from "what enhances output quality when available" (a real SMS send, a fluent LLM-written description) was a deliberate engineering decision carried from Review I into this implementation phase.

4. Knowledge Retrieval and Agentic RAG Design

The Risk Assessment and Dispatch agents need to turn a raw detection event — a sign label, a confidence score, and a timestamp — into a clear, caregiver-readable description of what is happening and where, without inventing medical detail that was not actually observed. Because this text feeds a safety-critical notification channel, the retrieval-and-generation pattern used here is deliberately narrow and grounded rather than an open-domain retrieval-augmented generation setup over a large external corpus.

Concretely, before any text is generated, the Risk Assessment agent assembles a structured context object: the detected sign or fallback event and its confidence, the current escalation stage, the room and caregiver profile on file, and the relevant recent entries from that room's own event log. This structured context — not a free web search — is what is passed to the reasoning model (DeepSeek API, or a locally hosted Qwen3 model served through Ollama when no external API key is configured) to produce a natural-language summary suitable for an SMS notification or an emergency-services description. If no reasoning-model credentials are available at all, the same structured context is instead filled into a fixed, pre-written template, guaranteeing that the escalation pipeline's ability to notify someone never depends on a generative model behaving reliably — generation only affects the fluency of the message, never whether or when it is sent.

This is intentionally a lightweight, project-scoped retrieval pattern: the "knowledge source" retrieved from is the system's own structured caregiver and event records rather than a large indexed corpus, which is an appropriate match for a narrow, five-sign safety vocabulary where hallucinated or under-

grounded detail carries real risk. Preliminary testing during this review confirms that grounding the model strictly in this structured context, and always retaining the deterministic template as a fallback, keeps generated descriptions consistent with what was actually detected. Extending this into a broader knowledge base — for example, indexed caregiver-specific instructions or condition-specific response guidance — is identified as future work rather than a Review II deliverable, given the added grounding and validation work such an extension would require.

5. Preliminary Evaluation Results and Comparison with Baseline Solutions

5.1 Recognition Model Evaluation

The perception classifier — a stacked LSTM network (64 units with return sequences, dropout 0.3, followed by a second LSTM of 128 units, dropout 0.3, a 64-unit fully connected ReLU layer, and a softmax output) — was trained on an 80/20 stratified train-test split using the Adam optimiser and categorical cross-entropy loss, with a batch size of 8 and early stopping (patience 15 epochs) to suit the modest size of the purpose-recorded dataset. Table below summarises the current evaluation snapshot.

Metric	Value
Number of distress classes	5 (Heart-pain, Help, Pain, Cant-breathe, Dizzy)
Input sequence shape	30 × 258 (landmarks per frame)
Train / test split	80% / 20% (stratified)
Final test accuracy	96.88%

This result is favourable relative to accuracy figures typically reported for general-vocabulary ISL recognition using comparable MediaPipe-plus-LSTM pipelines, which tend to fall in the 85–95% range on larger, more varied sign vocabularies. This is attributed primarily to the smaller number of classes and the deliberately high inter-class visual distinctiveness of the chosen distress vocabulary, rather than to any inherent superiority of the model architecture — a distinction the team is treating as an explicit limitation rather than an achievement to be over-claimed. Qualitatively, signs with visually distinct hand shapes and motion trajectories (for instance, Cant-breathe versus Dizzy) were separated with very high confidence during informal testing, while signs sharing more similar hand configurations showed occasional confusion, though this did not affect the reported test-set accuracy.

5.2 Comparison with Baseline System Categories

Because SignAlert sits at the intersection of two existing research and product categories — sign-language recognition systems and vision-based fall-detection systems — its preliminary evaluation also includes a qualitative comparison against those two baseline categories, rather than accuracy alone, since accuracy figures are not directly comparable across different vocabularies and tasks.

Property	Prior ISL Recognition Systems	Prior Fall-Detection Systems	SignAlert (this work)
Vocabulary scope	Broad, general-purpose	Not applicable	Narrow, safety-critical

Property	Prior ISL Recognition Systems	Prior Fall-Detection Systems	SignAlert (this work)
Non-signing events (falls/stillness)	Not covered	Covered	Covered
False-positive filtering	Not typically addressed	Varies by system	Explicit confirmation layer
Downstream action	None (text/speech output only)	Alert only	Staged notification / escalation
Deployment cost	Varies	Requires wearable/dedicated sensor	Low (camera + free-/trial-tier SMS)

The comparison shows the deliberate trade-off at the centre of the project's design: prior ISL recognition systems achieve comparable or higher accuracy on much larger vocabularies but stop at producing a text label, and prior fall-detection systems close the loop on a single event type but have no concept of intentional, signed communication. SignAlert trades vocabulary breadth for depth of downstream action — a narrower set of triggers, each one carried through to a real notification rather than a passive flag — which is the axis this review is measuring progress against, alongside recognition accuracy.

5.3 Implementation Progress Summary

Mapped against the deliverables expected at this stage of the project, implementation progress can be summarised as follows.

Deliverable Area	Status	Evidence
Functional prototype	Working end to end	Live camera → perception → fallback → confirmation → escalation → Twilio dispatch, plus a status dashboard.
Agent orchestration workflow	Implemented	LangGraph state machine with seven agents, one instance per monitored room.
Tool integrations	Implemented (with fallbacks)	MediaPipe, LSTM classifier, FastAPI/WebSocket, Twilio, DeepSeek/Qwen3, all wired and independently testable.
Agentic RAG implementation	Implemented at scoped level	Structured event/caregiver context assembled and passed to the reasoning model, with deterministic template fallback.
Memory and state management	Implemented	Per-room state object; escalation state persists independent of client connection.
Preliminary evaluation results	Completed for perception model	96.88% test accuracy on the five-class distress vocabulary (80/20 stratified split).
Comparison with baseline solutions	Completed (qualitative)	Benchmarked against prior ISL-recognition and fall-detection system categories on vocabulary, coverage, filtering, action, and cost.

6. Current Limitations and Next Steps

- Dataset scale and signer diversity: the training set was recorded by a limited number of signers, so accuracy under this evaluation does not yet guarantee equivalent performance across signers with different style, speed, or physical characteristics.
- Vocabulary breadth: the five-class vocabulary demonstrates the architecture but is narrower than a deployed system would need; expanding it is expected to reduce accuracy somewhat, consistent with trends in the wider ISL recognition literature.
- Empirical validation of timeouts: confirmation-window and escalation timeout values were chosen from reasonable design judgment rather than a formal user study; a structured study with Deaf and non-verbal participants and caregivers is planned to validate these against real false-alarm tolerance.
- Notification delivery constraints: reliance on a Twilio trial account restricts SMS delivery to pre-verified numbers; production deployment will require an upgraded messaging account and, where regionally available, a direct emergency-dispatch integration.
- Retrieval scope: the current knowledge context for the reasoning agent is limited to the system's own structured records; broadening this into an indexed, caregiver-specific knowledge base is planned future work.
- Reasoning-model dependency at the margins: while the escalation logic never depends on the LLM being available, the fluency and clarity of the caregiver-facing description currently does; a larger bank of pre-written templates per event type would reduce this gap further.

Heading into the final project phase, the immediate priorities are to run the structured user study referenced above, replace the pose-based fallback heuristics with a learned model once sufficient labelled fall/stillness data specific to the deployment environment is available, and move beyond a Twilio trial account toward production-grade messaging. Each of these next steps was already identified as future work in the original system design and is now sequenced against the milestones for the end-semester viva.

7. Conclusion

Project Review II moves SignAlert from an architectural proposal into a working, end-to-end functional prototype: the perception, fallback, confirmation, risk-assessment, escalation, and dispatch agents are implemented and orchestrated through a LangGraph state machine with per-room memory and state management that survives disconnection of the monitoring client, all tools identified in Review I are integrated into a running FastAPI-and-WebSocket backend with a live dashboard, and a scoped, grounded retrieval pattern feeds the reasoning layer without making the escalation logic dependent on generative reliability. Preliminary evaluation shows a strong 96.88% test accuracy on the five-class distress vocabulary and a favourable qualitative position relative to both prior ISL-recognition and prior fall-detection baselines, while the limitations identified above — dataset diversity, vocabulary breadth, timeout validation, and notification-account constraints — define the concrete next steps heading into the final project phase.